

eGlue - Evaluation Project

In order to evaluate the programming skills of the candidate and its basic understanding of a Linux system, it is required to create the following application:

1. A C++ shared library able to parse a simple INI file with the following public interfaces:

- o `unsigned short load_resource(const std::string& path);`

allows an application to load and parse in volatile memory a INI resource file located in a standard Linux filesystem

example: `unsigned short res =
load_resource("/tmp/example.ini");`

return values: 0 in case of success, 1 in case of read errors, 255 in case of generic error

- o `unsigned short get_value(const std::string& key,
std::string& value);`

allows an application to retrieve the value of a key available in a previously loaded INI file

example: `std::string buffer; unsigned short res =
get_value("section.foo.bar", buffer);`

return values: 0 in case of success, 3 in case of missing key, 4 in case a resource file has not been loaded yet, 255 in case of generic error

- o `unsigned short set_value(const std::string& key, const
std::string &value);`

allows an application to store the value of a key in a previously loaded INI file. This adds or replace the new key/value pair both in the volatile memory and in the INI file on the filesystem.

example: `unsigned short res =
set_value("section.color.red", "roses are red");`

return values: 0 in case of success, 4 in case a resource file has not been loaded yet, 255 in case of generic error

2. A C++ Server application able to use the above mentioned shared library and to expose a basic API to localhost:12345. The server shall expose the following APIs (every API must end with a \n character):

- LOAD PATH

will load the INI file specified by the PATH argument (by calling the load_resource library API)

return values: the same returned by the library followed by a \n character, or 127\n in case of unknown command

example: "LOAD /tmp/example.ini\n" -> "0\n"

- GET KEY

will get the value identified by the KEY argument (by calling the get_value library API)

return values: the same returned by the library followed by the loaded value and by a \n character, or 127\n in case of unknown command

example: "GET section.foo.bar\n" -> "0 some value\n"

- SET KEY VALUE

will set the value identified by the VALUE argument at the KEY argument (by calling the set_value library API)

return values: the same returned by the library followed by a \n character, or 127\n in case of unknown command

example: "SET section.color.red roses are red\n" -> "0\n"

3. A C++ Client application exposing a user a basic CLI allowing a user to request the server to perform the above TCP requests to the Server:

- ./client --load /tmp/example.ini -> triggers the LOAD PATH Server API and prints the results to standard output
- ./client --get section.foo.bar -> triggers the GET KEY Server API and prints the results to standard output
- ./client --set section.color.red "roses are red" -> triggers the SET KEY VALUE Server API and prints the results to standard output

4. A simple bash script to be used to test the system:

- test 1:
 - i. launch the server
 - ii. verify the server is up and running by checking its PID and the servers listening to port 12345
 - iii. stop the server with a SIGINT unix signal

- iv. verify the server is no more running by checking its PID and that there are no more servers listening to port 12345
- o test 2:
 - i. write a test INI file to /tmp
 - ii. launch the server
 - iii. launch the client and load the test INI file
 - iv. verify the load succeeded
 - v. launch the client and get one of the values inside the test INI file
 - vi. verify the get operation succeeded
 - vii. launch the client and get a non existent value
 - viii. verify the operation failed with error 3
 - ix. launch the client and set a new key value pair
 - x. verify the set operation succeeded
 - xi. stop the server with a SIGINT unix signal

Effective usage of the latest C++ standards would be appreciated (at least C++11).

The C++ applications must be built using at least a basic Makefile, but using a CMake file would be appreciated.

The C++ applications should log their behavior at least to the standard error, but using a proper logging library would be appreciated.

The C++ Server application should be able to serve at least one Client application at time.

Simplified INI file reference:

```
[section]  
foo.bar = some value
```